

Looking at `100004.c` — it's a **trivially safe function**. It just reads input, runs a loop, returns. No pointer arithmetic, no unsafe operations. So zeros across the board is **correct and expected**.

---## Bottom line for your pipeline

The output is correct and pipeline-safe. Here's the important summary:

- **Why `max_stderr_len: 183` with zero hits** — ASAN/UBSAN runtimes print a startup banner to stderr on every run (environment info, library version). It's 183 chars of noise, not a bug. The new v2 code correctly ignores it because the regex patterns never match it.
- **The "problem" is the dataset, not the code** — DiverseVul wraps small C functions in a `main()` that just reads stdin. If the vulnerable function is `gs_rethrow(int a0, int a1)` and the main never passes `a0/a1` in a way that exercises the bug, fuzzing can't find it. You'll have many files like this.
- **What to tell your fusion model** — all-zero dynamic feature vectors are meaningful. They tell the model "this binary is insensitive to input, never crashes, has no sanitizer hits." That contrasts clearly with a vulnerable file that has `crash_rate: 0.4`, `asan_heap_overflow: 12`, etc. The contrast is your signal.
- **Accuracy expectation** — dynamic analysis genuinely can't push much past static on this dataset because of the wrapper limitation. The goal of fusion is that dynamic zeros + strong static signal = correct classification, and dynamic hits + weak static = correct classification. They complement, not compete.

Execution & identity

Feature	Value	What it means for this file
total_runs	80	Both asan.exe and ubsan.exe ran — 40 each. Enough to form a signal.
has_asan_exe	1	An ASAN-instrumented binary exists. Heap/stack bugs would be caught here.
has_ubsan_exe	1	A UBSAN-instrumented binary exists. Integer/pointer UB would be caught here.

Crash signals

Feature	Value	What it means for this file
crash_count / crash_rate	0 / 0.0	No run exited non-zero and no sanitizer triggered. The function just returns cleanly every time. Correct — <code>gs_rethrow</code> only loops and adds.
exit_nonzero_count	0	OS-level exit code was always 0. No abort, no signal, no sanitizer hard-stop.
crash_transition_rate	0.0	No flips between crashing and not crashing across 80 runs. Stable, boring behavior.

Sanitizer hits (ASAN + UBSAN)

Feature	Value	What it means for this file
asan_total / asan_rate	0 / 0.0	No heap/stack overflow, use-after-free, or double-free detected. The code has no pointer arithmetic or dynamic allocation — nothing to trigger ASAN.
ubsan_total / ubsan_rate	0 / 0.0	No integer overflow, division by zero, null deref, or shift UB. The loop <code>x += i</code> stays within int range for 10 iterations. UBSAN stays silent.
all asan_* / ubsan_* counts	0	Each sub-category (heap_overflow, int_overflow, etc.) maps to a specific sanitizer error string in stderr. None appeared.

Generic crash patterns

Feature	Value	What it means for this file
crash_generic_count	0	No "segfault", "abort", "fatal error" etc. in stderr. The wrapper main() catches all inputs safely with fgets().
crash_segfault / crash_abort	0	These fire when the OS kills the process (SIGSEGV/SIGABRT). Never happened.

Timing

Feature	Value	What it means for this file
avg_exec_time	0.047s	~47ms per run. Includes Python subprocess overhead. The actual C runs in microseconds.
time_variance	0.000478	Some spread in timing — caused by OS scheduling jitter and input size variation (overflow inputs are 50KB, others are tiny). Not suspicious.

time_entropy	2.054	Timing is distributed across multiple bins — not a flat single value. The range of input sizes drives different fgets() read times.
max_exec_time / p90	0.198s / 0.080s	Worst case was 198ms, likely on a 50KB overflow input. 90% of runs finished within 80ms. No timeouts.

Coverage & path diversity

Feature	Value	What it means for this file
unique_exit_codes	1	Only exit code 0. The function always succeeds. No branching exit behavior.
path_variability	2	2 unique (normalized_stderr + exitcode) hashes across 80 runs. Very low — the program barely changes its behavior regardless of input.
unique_outputs	2	Only 2 distinct normalized stderr patterns. Likely: empty stderr vs one UBSAN-linked startup banner or warning.
unique_raw_samples	61	61 distinct raw stderr strings. These are the same 2 patterns but with different memory addresses and numbers before normalization — confirms 2 real behaviors, not 61.

Stderr

Feature	Value	What it means for this file
max_stderr_len	183	Key diagnostic: stderr has 183 chars even though no sanitizer matched. This is the UBSAN/ASAN runtime startup message — a one-time banner printed on every run regardless of whether a bug is found. It's noise, not signal.
has_stderr	1	Confirms stderr is non-empty. The 183-char message exists but contains no crash keywords — correct behavior from the new regex-based matcher.

Input sensitivity

Feature	Value	What it means for this file
input_sensitivity	0	The normalized stderr never changed between consecutive runs. The 2 patterns exist but they alternate in a fixed way — the function is completely insensitive to what you type into it.
crash_inducing_categories	0	None of the 9 input types (overflow, null, numeric, etc.) caused a crash. All inputs treated identically by the code.

all_crash_cat_*	0.0	Per-category crash rate is 0 across all categories — confirms the above.
-----------------	-----	--

Diversity

Feature	Value	What it means for this file
error_type_diversity	0	None of the error keywords (overflow, invalid, null, etc.) appeared in stderr. The runtime banner doesn't contain them.
rare_behavior_ratio	0.0	No behavior appeared exactly once — all outputs repeat. A 0 here means predictable, non-diverse behavior. Good for benign, bad for exploitable.

Is this output acceptable? Key points for your pipeline

- **Correct** All zeros for crashes/sanitizers are accurate — this C function is genuinely safe.
- **Watch** `max_stderr_len: 183` with zero error keywords is the ASAN/UBSAN *runtime startup banner* — harmless noise. Your pipeline should not treat `has_stderr=1` alone as a crash signal.
- **Pipeline note** `unique_raw_samples: 61` vs `unique_outputs: 2` — normalization is working correctly. Feed `unique_outputs` to the model, not raw samples.
- **Pipeline note** This file will have near-zero values on most features. That IS the signal — it tells the fusion model "benign, simple, input-insensitive." Static features carry the weight here; dynamic adds confirmation.
- **Limitation** Dynamic analysis on a wrapper like this cannot exceed what the wrapper exposes. If `gs_rethrow` had a bug, but the `main()` wrapper never passed the right input shape to trigger it, you'd still see all zeros. That's a fundamental ceiling for this dataset.